

Lab 1: Struct a class

Explicació de la solució

1. El salt al paradigma orientat a objectes

A PRO1, definíem un **struct** i després fèiem funcions passant la tupla com a paràmetre **funcio(tupla)**.

Ara, es tracta de fer que aquestes “funcions” visquin **a dins** del **struct** (mètodes), de manera que tinguin **accés directe a l'estat intern**. Canviarem la sintaxi per **tupla.funcio()**, encapsulant finalment el codi usant la protecció que dona **class**.

Com programàvem a PRO1 (data.cc)

```
“cpp [data.cc] struct Data { int dia, mes, any; };
Data data_suma_dies(const Data& d, int dies) { Data res = d; res.dia += dies; // ... return res; }
int main() { Data avui = data_actual(); Data dema = data_suma_dies(avui, 1); }
```

Com programarem a PRO2

La nostra ``Data`` té ****responsabilitats pròpies****. Només permet l'accés a variables i funcions públiques.

```
```cpp [main.cpp]
class Data {
private:
 int dia, mes, any;

public:
 Data() { dia = 1; mes = 1; any = 2000; }

 Data suma_dies(int dies) const {
 Data resultat = *this;
 resultat.dia += dies;
 return resultat;
 }
};

int main() {
 Data avui;
 Data dema = avui.suma_dies(1);
}
```

#### Solució 1: Conversió a classe data.cc

*[Simulació interactiva disponible a la web]*

### Arxiu original racional.cc

```
“cpp [racional.cc] struct Racional { int num_, den_; };

int mcd(int a, int b) { while (b != 0) { int aux = b; b = a % b; a = aux; } return a; }

int signe(int n) { return n < 0 ? -1 : 1; }

Racional racional_llegeix() { char ; Racional r; cin » r.num » _ » r.den_; return r; }

Racional racional_crea(int n, int d) { if (n == 0) { return {0, 1}; } int sig = signe(n) * signe(d); n = abs(n);
d = abs(d); const int m = mcd(n, d); return {sig * n / m, d / m}; }

Racional racional_suma(Racional a, Racional b) { return racional_crea(a.num_ * b.den_ + b.num_ *
a.den_, a.den_ * b.den_); }

Racional racional_resta(Racional a, Racional b) { return racional_crea(a.num_ * b.den_ - b.num_ * a.den_,
a.den_ * b.den_); }

Racional racional_multiplica(Racional a, Racional b) { return racional_crea(a.num_ * b.num_, a.den_ *
b.den_); }

Racional racional_divideix(Racional a, Racional b) { return racional_crea(a.num_ * b.den_, a.den_ *
b.num_); }

void racional_escriu(Racional a) { cout « a.num_; if (a.den_ > 1) { cout « ‘/’ « a.den_; } }

int main() { Racional acum = racional_llegeix(); racional_escriu(acum); cout « endl;

string op;
while (cin >> op) {
 Racional r = racional_llegeix();
 if (op == "+") {
 acum = racional_suma(acum, r);
 } else if (op == "-") {
 acum = racional_resta(acum, r);
 } else if (op == "*") {
 acum = racional_multiplica(acum, r);
 } else if (op == "/") {
 acum = racional_divideix(acum, r);
 }
 racional_escriu(acum);
 cout << endl;
}

} “
```

### Solució 2: Conversió a classe racional.cc

*[Simulació interactiva disponible a la web]*